

SIMATS ENGINEERING
ASSESSMENT TOOL 3

NAME : B . PANEESHA

REG. NO: 192472206

COURSE NAME: OPERATING SYSTEMS

COURSE CODE: CSA04

COURSE OUTCOME COVERED

CO2: Analyze process management and deadlock handling techniques to improve CPU utilization and system performance(BL3-BL4)

Assessment Tool Weightage: 30%

QUESTIONS: 10

Total Marks:50

Annexure A

Debugging or Optimization task

Code of Conduct:

I am B. Paneesha (192472206) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: B. Paneesha.

Q1. Deadlock Debugging (Code Trace)

You are given the following pseudo-code for two processes sharing resources R1 and R2:

Process P1:

```
lock(R1)
wait(100 ms)
lock(R2)
critical section
unlock(R2)
unlock(R1)
```

Process P2:

```
lock(R2)
wait(100 ms)
lock(R1)
critical section
unlock(R1)
unlock(R2)
```

Identify the exact condition that causes deadlock. Suggest an optimisation to eliminate the deadlock without changing process logic.(10

Q1. Deadlock Debugging (10 Marks

Given

Process P1

```
lock(R1)
wait(100 ms)
lock(R2)
Critical Section
```

unlock(R2)

unlock(R1)

Process P2

lock(R2)

wait(100 ms)

lock(R1)

Critical Section

unlock(R1)

unlock(R2)

What happens?

Time	Process P1	Process P2
t1	Locks R1	Locks R2
t2	Waits 100 ms	Waits 100 ms
t3	Tries to lock R2 (busy)	Tries to lock R1 (busy)

Now,

- **P1 holds R1** and waits for **R2**
- **P2 holds R2** and waits for **R1**

Neither process can continue.

Exact Cause of Deadlock

This happens because of **Circular Wait**.

Four deadlock conditions are present:

1. Mutual Exclusion ✓ (Resources cannot be shared)
2. Hold and Wait ✓ (Each process holds one resource and waits for another)
3. No Preemption ✓ (Resources cannot be forcibly taken)
4. Circular Wait ✓ (**Main reason**)

Optimization

Acquire resources in the **same order**.

Example:

Both P1 and P2 should use:

lock(R1)

lock(R2)

Critical Section

unlock(R2)

unlock(R1)

Since both request resources in the same order, circular wait cannot occur.

Q2. CPU Utilisation Problem

A system shows low CPU utilisation (30%) even though processes are ready. You analyse the scheduler log and find frequent transitions:

ready → waiting → ready → waiting → ready ... Debug the cause using process management concepts and propose a system-level optimisation to improve utilisation.

Q2. CPU Utilisation Problem (10 Marks)

Given

CPU Utilization = **30%**

Process states:

Ready → Waiting → Ready → Waiting → Ready

Debugging

Processes are frequently moving from **Ready** to **Waiting**.

Waiting state means the process is waiting for:

- Disk
- Keyboard
- Network
- Printer
- Other I/O devices

Therefore,

CPU becomes idle while processes wait for I/O.

This is called an **I/O-bound workload**.

Why CPU Utilization is Low

Example:

CPU executes process for 2 ms

↓

Process requests disk

↓

Moves to Waiting

↓

CPU becomes idle

↓

Process returns to Ready

↓

Again waits for I/O

Most time is spent waiting rather than executing.

Optimization

1. Increase Multiprogramming

Load more processes into memory.

When one process waits for I/O, another process uses the CPU.

2. Improve I/O Performance

Examples:

- SSD instead of HDD
- DMA (Direct Memory Access)
- Asynchronous I/O
- Better buffering

Result:

- Less waiting
- Higher CPU utilization

Q3. Memory Bottleneck Debugging

A multi-threaded program frequently encounters page faults, slowing performance. The memory

map shows:

- Code: 5 MB
- Heap: 150 MB
- Stack: 1 MB per thread • 25 threads running • Physical RAM available: 2 GB

Identify the memory issue causing performance degradation and propose two optimisations.

Q3. Memory Bottleneck Debugging (10 Marks)

Given

Code = 5 MB

Heap = 150 MB

Stack = 1 MB per thread

Threads = 25

RAM = 2 GB

Memory Calculation

Stack Memory

$25 \times 1 \text{ MB} = 25 \text{ MB}$

Total Memory

Code = 5 MB

Heap = 150 MB

Stacks = 25 MB

Total = 180 MB

Although total memory fits into RAM, performance is poor because many threads access different memory pages.

Root Cause

Many threads access different pages.

This reduces locality of reference.

As a result:

- Frequent page faults
- Frequent disk access
- Slow execution

Optimization 1

Reduce the number of threads.

Instead of 25 threads,

use a **Thread Pool**.

Optimization 2

Improve memory locality.

Examples:

- Use contiguous memory
- Reduce heap fragmentation
- Reuse objects

Result:

- Fewer page faults
- Better performance

Q4. Starvation Debugging

You observe that process P5 never gets CPU time in a multilevel queue scheduler:

- Q1 (Highest Priority): Interactive
- Q2: System
- Q3: Batch
- Time Quantum: Q1 = 5ms, Q2 = 10ms, Q3 = FCFS

P5 is in Q3 but keeps getting postponed. Debug the root cause of starvation and suggest an optimisation to ensure fairness.

Q4. Starvation Debugging (10 Marks)

Given

Queue	Priority
Q1	Interactive
Q2	System
Q3	Batch (P5)

Scheduler always serves:

Q1 first

↓

Q2

↓

Q3

Root Cause

If Q1 always has processes,

Q3 never gets CPU.

Therefore,

P5 waits forever.

This is called **Starvation**.

Optimization

Use **Aging**.

Example

Every process waiting for a long time receives a higher priority.

P5 waits

↓

Priority increases

↓

Moves to Q2

↓

Eventually reaches Q1

↓

Gets CPU

Result:

Every process eventually executes.

Q5. Context-Switch Overhead Debugging

A server performs 20,000 context switches/sec, causing high overhead. Log details show:

- Average burst time per thread: 2ms
- I/O wait frequency: very high
- Preemptive scheduler with quantum = 1ms

Explain why context switching is so high and propose two optimisations to reduce overhead.

Q5. Context-Switch Overhead Debugging (10 Marks)

Given

Context switches = 20,000/sec

Quantum = **1 ms**

Burst Time = **2 ms**

High I/O frequency

Why Context Switching is High

Each thread needs 2 ms to finish.

Scheduler interrupts every 1 ms.

Example

Run 1 ms

↓

Context Switch

↓

Run remaining 1 ms

↓

Context Switch

Every thread requires at least two context switches.

Frequent I/O creates even more switches.

Optimization 1

Increase Time Quantum

Example

1 ms

↓

4 ms

Most threads finish before preemption.

Optimization 2

Reduce Thread Switching

Use

- Thread Pool
- Event-driven server

Benefits

- Less scheduling
- Less overhead

Q6. Semaphore Synchronization Debugging

A producer-consumer application occasionally hangs during execution. The semaphore values

observed are:

mutex = 1

empty = 0

full = 10

Buffer size = 10

Producer Code:

wait(mutex)

wait(empty)

produce item

signal(full)

signal(mutex)

Consumer Code:

wait(mutex)

wait(full)

consume item

signal(empty)

signal(mutex)

Identify the synchronization bug causing the system to hang. Explain which deadlock condition is being violated. Suggest an optimization to ensure correct synchronization without changing the producer- consumer functionality.

Q6. Semaphore Synchronization Debugging

Given

mutex = 1

empty = 0

full = 10

Buffer is completely full.

Producer

wait(mutex)

wait(empty)

produce

signal(full

signal(mutex)

Consumer

wait(mutex)

wait(full)

consume

signal(empty)

signal(mutex)

Root Cause

Producer locks **mutex** first.

Then executes

wait(empty)

Since

empty = 0

Producer blocks.

But producer still holds **mutex**.

Consumer cannot enter because mutex is locked.

System hangs.

Deadlock Condition

This violates **Hold and Wait**.

Producer:

- Holds mutex
- Waits for empty

Consumer:

- Waits for mutex

No one proceeds.

Correct Solution

Producer

wait(empty)

wait(mutex)

produce

signal(mutex)

signal(full)

Consumer

wait(full)

wait(mutex)

consume

signal(mutex)

signal(empty)

This prevents deadlock.

Q7. Thrashing Detection and Optimization

A virtual memory system reports the following statistics:

Number of processes: 50

Average page fault rate: 250 faults/sec

CPU Utilization: 18%

Disk Utilization: 95%

Free Frames Available: 2%

The operating system suspects thrashing. Debug the root cause of the performance degradation. Explain the relationship between page faults and CPU utilization. Propose two operating system optimizations to reduce thrashing.

Q7. Thrashing Detection

Given

Processes = 50

Page faults = 250/sec

CPU = 18%

Disk = 95%

Free Frames = 2%

Root Cause

Too many processes compete for too few memory frames.

Operating system constantly swaps pages.

This is called **Thrashing**.

Relationship

More Page Faults

↓

More Disk Access

↓

CPU waits

↓

CPU Utilization decreases

Hence,

High page faults → Low CPU utilization.

Optimization 1

Reduce Degree of Multiprogramming.

Run fewer processes.

Each process gets more memory.

Optimization 2

Use

- Working Set Model
- Page Fault Frequency (PFF)

These allocate enough frames to active processes.

Result:

- Fewer page faults
- Better CPU utilization

Q8. File System Performance Debugging

A database server experiences slow file access times. System logs indicate:

Average file size: 500 MB

Disk seeks per request: 120

Disk utilization: 92%

Cache hit ratio: 15%

Identify the file system bottleneck affecting performance. Explain why disk access latency is increasing. Suggest two optimizations to improve file access efficiency.

Q8. File System Performance Debugging

Given

File size = 500 MB

Disk seeks = 120/request

Disk utilization = 92%

Cache hit = 15%

Root Cause

Files are fragmented.

Many disk seeks are required.

Very low cache hit ratio causes repeated physical disk access.

Why Access is Slow

Disk access time

Seek Time

+

Rotational Delay

+

Transfer Time

Since seek operations are very high,

latency increases.

Optimization 1

Improve Cache

Increase cache size.

Use read-ahead buffering.

Optimization 2

Reduce Fragmentation

Use

- Contiguous allocation
- Extent-based allocation

Result:

- Fewer seeks
- Faster file access

Q9. Thread Scheduling Debugging

A web server creates one thread per client request. During peak hours:

Active Threads: 5,000

CPU Utilization: 60%

Memory Usage: 92%

Average Response Time: 12 seconds

Scheduler logs show frequent thread creation and termination. Identify the primary issue affecting server performance. Explain how thread management contributes to the delay. Recommend two optimizations to improve scalability and response time.

Q9. Thread Scheduling Debugging

Given

Threads = 5000

CPU = 60%

Memory = 92%

Response = 12 sec

Frequent thread creation

Root Cause

One thread is created for every client request.

This causes

- Large memory usage
- Context switching
- Scheduling overhead

Why Delay Occurs

Every thread requires

- Stack memory
- Scheduler entry
- Creation
- Destruction

Thousands of threads waste CPU time.

Optimization 1

Use a **Thread Pool**.

Reuse existing threads.

Optimization 2

Use **Asynchronous I/O** or an event-driven model.

Benefits

- Fewer threads
- Less memory
- Faster response

Q10. Disk Scheduling Optimization Problem

A disk scheduling log shows the following request queue:

98, 183, 37, 122, 14, 124, 65, 67

Current Head Position: 53

The system uses FCFS scheduling and reports:

Average seek time: Very High

Throughput: Low

User complaints regarding slow response Debug why FCFS is causing poor performance in this

scenario. Compare FCFS with one suitable disk scheduling algorithm for this workload. Propose an optimization to improve seek time and overall throughput.

Q10. Disk Scheduling Optimization

Given

Queue

98, 183, 37, 122, 14, 124, 65, 67

Current Head = **53**

FCFS is used.

Why FCFS is Poor

Head movement

53

↓

98

↓

183

↓

37

↓

122

14

↓

124

↓

65

↓

67

The disk head moves back and forth over long distances.

Result

- High seek time
- Low throughput
- Slow response

Better Algorithm – SSTF

SSTF always selects the **closest request**.

Example order:

53

↓

65

↓

67

↓

37

↓

14

↓

98

↓

122

↓

124

↓

183

This greatly reduces total head movement.

Optimization

Replace **FCFS** with **SSTF** (or **SCAN/LOOK** if fairness is also important).